

ELEC520

Python programming

Summer 2026

tuple, set, dictionary

https://colab.research.google.com/drive/15hxlDyRdmxWklGP71JqbA_kxYDhmAudr?usp=sharing

Goal

- 어떤 자료형(자료구조)을 사용할까?

문자열1: Hello, world!
문자열2: how are you?
공통 문자: d l
문자열1에만 출현한 문자: d l

달: 7
July

달: 13
해당 달은 존재하지 않습니다.

문장: Apple: "Papa!"
출현한 알파벳: e l a p

Remark.

튜플은 리스트와 유사: 순서 중요, 중복 허용
그럼 차이점은?

튜플tuple

생성과 멤버 접근

패킹과 언패킹

연산: +, *, 슬라이싱

제공되는 built-in ft. 과 메소드: count(), index()

언제 사용하나?

튜플

- 여러 개의 요소(항목값)를 가지는 컬렉션 자료형 **collection data type**
- 교환불가능 **immutable** 속성
 - 튜플 내부의 객체를 변경하거나 삭제하는 것도 불가능하다는 특징을 가짐
- 리스트와는 달리 한 번 정해지면, 내용을 바꿀 수 없다.
 - 단점: 따라서 사용할 수 있는 메소드도 제한적
 - 장점: 리스트보다 메모리 절약, 속도 빠름
- `t = ('one', 'two', 'three')` 와 같이 괄호로 둘러싸인 형태
- 언제 사용하나?

튜플을 생성하는 여러 가지 방법

빈 튜플 만들기	<code>tuple0 = ()</code>
하나의 요소를 가진 튜플 만들기	<code>tuple1 = (1,)</code> # 반드시 쉼표를 사용해야 함에 유의
기본적인 튜플 만들기	<code>tuple2 = (1, 2, 3, 4)</code>
간단한 방식의 튜플 만들기	<code>tuple3 = 1, 2, 3, 4</code>
리스트로부터 튜플 만들기	<code>n_list = [1, 2, 3, 4]</code> <code>tuple4 = tuple(n_list)</code>

대화창 실습 : 튜플의 생성과 조회

```
>>> t = (1, 2, 3, 4)
```

```
>>> t[0]    # 튜플 t의 첫번째 항목을 조회 함
```

```
1
```

```
>>> t[1]
```

```
2
```

성분이 하나인 튜플

1) 잘못된 튜플 선언 :

```
>>> tup = (100)
>>> tup          # 정수형 100
100
>>> type(tup)
<class 'int'>
```

2) 올바른 튜플 선언 :

```
>>> tup = (100,)
>>> tup          # 튜플형 (100,)
(100,)
>>> type(tup)
<class 'tuple'>
```

Immutable

- 튜플은 리스트와는 달리 내부의 값을 변경할 수 없으므로 다음과 같은 개별 항목에 대한 할당 연산은 오류를 발생시킴.
 - 이와 같은 속성을 교환 불가능 immutable속성이라 한다.

대화창 실습 : 튜플 항목에 대한 할당 연산은 오류를 발생시킴

```
>>> t = (0, 1, 2, 3, 4)
```

```
>>> t[0] = 100    # 할당연산은 오류를 발생시킴
```

```
...
```

```
TypeError: 'tuple' object does not support item assignment
```

튜플의 패킹, 언패킹

- 패킹 **packing**
 - 하나의 변수에 여러 개의 값을 넣는 것을 의미
- 언패킹 **unpacking**
 - 패킹된 변수가 있으면, 여기에서 여러 개의 값을 꺼내오는 것

[표 6-11] 패킹과 언패킹

패킹	언패킹
<pre>>>> a = (1, 2) # 튜플 패킹 >>> a[0] # 튜플 항목에 대한 참조 1 >>> a[1] 2</pre>	<pre>>>> c = (3, 4) # 튜플 패킹 >>> d, e = c # 튜플 언패킹 >>> d 3 >>> e 4</pre>



LAB 6-6 : 튜플의 생성과 패킹, 언패킹

1. (1919, 3, 1)의 요소 값을 가지는 the_day 튜플을 생성하여라. 그리고 이 튜플의 요소를 언패킹을 통해 year, month, day 변수에 할당한 후 다음과 같이 출력하여라.

1919년 3월 1일은 삼일절입니다.

2. 10, 20, 30의 순서대로 이루어진 리스트를 튜플로 변환하여 a, b, c에 언패킹한 후 이를 다음과 같이 출력하여라.

a = 30

b = 20

c = 10

튜플의 연산: +(연결), *(반복)

대화창 실습 : 튜플의 +와 * 연산

```
>>> t0 = (10, 20, 30)
```

```
>>> t1 = t0 + t0  # 튜플의 결합 연산
```

```
>>> t1
```

```
(10, 20, 30, 10, 20, 30)
```

```
>>> t1 = t0 * 3  # 튜플의 반복 연산
```

```
>>> t1
```

```
(10, 20, 30, 10, 20, 30, 10, 20, 30)
```

```
>>> t3 = t0 + 40  # 튜플과 정수는 덧셈 연산을 할 수 없음.
```

```
...
```

```
TypeError: can only concatenate tuple (not "int") to tuple
```

```
>>> t4 = t0 + (40, )  # 튜플과 하나의 요소를 가진 튜플의 덧셈 연산
```

```
>>> t4
```

```
(10, 20, 30, 40)
```

튜플의 메소드

메소드	하는 일
count(x)	튜플 내에 특정 값 x가 몇 개 있는가를 알려줌
index(x)	튜플 내의 특정 값 x의 위치를 알려줌

대화창 실습 : 튜플의 메소드 테스트

```
>>> t = (10, 20, 30, 20, 20, 10, 50)
```

```
>>> t.count(10)    # 튜플에 포함된 10 원소의 개수
```

```
2
```

```
>>> t.count(20)    # 튜플에 포함된 20 원소의 개수
```

```
3
```

```
>>> t.index(30)    # 튜플에 포함된 30 원소의 인덱스
```

```
2
```

```
>>> t.index(50)    # 튜플에 포함된 50 원소의 인덱스
```

```
6
```

튜플의 성분 변경



LAB 6-7 : 튜플의 활용

1. 20190010이라는 학번과 '홍길동'이라는 이름, 179라는 키를 튜플의 요소로 가지는 튜플 객체와 참조 변수 `person`을 생성하여라.

```
person = ('홍길동', 2019001, 179)
```

2. `person`의 학번을 2019003으로 변경하여라. 어떤 오류가 발생하는가?
3. `person`을 리스트로 변환하여 학번을 2020003으로 변경한 후 이를 다시 튜플로 변환하여 다음과 같이 출력하여라.

```
학번 변동 후 person = ('홍길동', 2020003, 179)
```

튜플의 슬라이싱

- 튜플 역시 문자열이나 리스트와 동일한 방식으로 슬라이싱이 가능
- 음수 인덱스를 지정할 경우 역순으로 가져오기 위하여 -1과 같은 음수 스텝을 사용

대화창 실습 : 튜플의 슬라이싱

```
>>> tu = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
>>> tu[1:5] # 튜플의 슬라이싱으로 1, 2, 3, 4의 네 원소를 가져온다
```

```
(1, 2, 3, 4)
```

```
>>> tu[0:10:3]
```

```
(0, 3, 6, 9)
```

```
>>> tu[::-1]
```

```
(9, 8, 7, 6, 5, 4, 3, 2, 1, 0)
```

집합set

생성
메소드

`add()`, `discard()`, `clear()`

`union()`, `intersection()`, `difference()`, `symmetric_difference()`

`issubset()`, `issuperset()`, `isdisjoint()`

언제 사용하나?

집합set

- 여러 개의 요소(항목값)를 가지는 컬렉션 자료형collection data type
- 순서가 없는 자료형
- 동일한 값을 가지는 항목이 중복이 허용되지 않음
- 교집합, 합집합, 차집합 등의 집합 연산 수행 가능

[표 6-13] 집합을 만드는 여러 가지 방법

빈 집합 만들기	<code>set0 = set()</code> # set0 = {} 와 같이할 경우 set0 는 dict 형이 됨
기본적인 집합 만들기	<code>set1 = {1, 2, 3, 4}</code>
튜플로부터 집합 만들기	<code>n_tuple = (1, 2, 3, 4)</code> <code>set2 = set(n_tuple)</code>
리스트로부터 집합 만들기	<code>n_list = [1, 2, 3, 4]</code> <code>set3 = set(n_list)</code>

- 집합을 만들 때는 다음과 같이 이미 생성된 리스트나 튜플에 set() 함수를 사용할 수도 있음
- 집합은 순서가 없는 원소의 모임이므로 **인덱스를 사용할 수 없음**

대화창 실습 : 리스트와 튜플로 부터 집합 만들기

```
>>> days_list = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'] # 리스트
>>> days_set = set(days_list) # 리스트로부터 집합 만들기
>>> days_set
{'Fri', 'Mon', 'Sat', 'Sun', 'Thu', 'Tue', 'Wed'}
>>> fruits_tuple = ('apple', 'orange', 'water melon') # 튜플
>>> fruits_set = set(fruits_tuple) # 튜플로부터 집합 만들기
>>> fruits_set
{'apple', 'orange', 'water melon'}
>>> h_str = 'hello' # 문자열
>>> h_set = set(h_str) # 문자열로부터 집합 만들기
>>> h_set # 집합에서는 문자 'l'의 중복을 허용하지 않음
{'h', 'l', 'e', 'o'}
```


집합의 메소드

메소드	하는 일
<code>add(x)</code>	집합 내의 특정 x 원소를 추가한다.
<code>discard(x)</code>	집합 내의 특정 x 원소를 삭제한다.
<code>clear()</code>	집합 내의 모든 원소를 삭제한다.
<code>union(s)</code>	s 집합과의 합집합을 구한다. $\cup$ 연산과 동일하다.
<code>difference(s)</code>	s 집합과의 차집합을 구한다. $-$ 연산과 동일하다.
<code>intersection(s)</code>	s 집합과의 교집합을 구한다. $\&$ 연산과 동일하다.
<code>symmetric_difference(s)</code>	s 집합과의 대칭 차집합을 구한다. $\wedge$ 연산과 동일하다.
<code>issubset(s)</code>	s 집합의 부분 집합인가를 구한다. True/False를 반환한다.
<code>issuperset(s)</code>	s 집합의 상위 집합인가를 구한다. True/False를 반환한다.
<code>isdisjoint(s)</code>	s 집합과 서로소인가를 구한다. True/False를 반환한다.

원소 추가 및 제거

대화창 실습 : 집합의 메소드 테스트

```
>>> s = {100, 100, 200, 200, 300, 400}    # 중복원소 100, 200이 있음
```

```
>>> s
```

```
{100, 200, 300, 400}
```

```
>>> s.add(500)    # 집합 s에 500 원소 추가
```

```
>>> s
```

```
{100, 200, 300, 400, 500}
```

```
>>> s.discard(100)    # 집합 s에 있는 100 원소를 삭제
```

```
>>> s
```

```
{200, 300, 400, 500}
```

집합 연산

1. 다음 두 집합이 있을 경우 이 집합에 대한 연산결과는 무엇인가?

$s1 = \{10, 20, 30, 40\}$

$s2 = \{30, 40, 50, 60, 70\}$

- 1) $s1 \mid s2$
- 2) $s1 \& s2$
- 3) $s1 - s2$
- 4) $s1 \wedge s2$
- 5) $s1.\text{issubset}(s2)$
- 6) $s1.\text{issuperset}(s2)$
- 7) $s1.\text{isdisjoint}(s2)$

- 집합 $s1$, $s2$ 와 그 연산의 결과

대화창 실습 : 집합과 연산

```
>>> s1 = {1, 2, 3, 4, 5, 6}
```

```
>>> s2 = {4, 5, 6, 7, 8, 9}
```

```
>>> s1 | s2    # 합집합을 구함
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
>>> s1 & s2    # 교집합을 구함
```

```
{4, 5, 6}
```

```
>>> s1 - s2    # 차집합을 구함
```

```
{1, 2, 3}
```

```
>>> s1 ^ s2    # 대칭 차집합을 구함
```

```
{1, 2, 3, 7, 8, 9}
```

- 집합 연산 $|$, $\&$, $-$, $\wedge$ 는 다음과 같이 `union()`, `intersection()`, `difference()`, `symmetric_difference()` 메소드를 사용하여 동일한 결과를 얻을 수 있음

대화창 실습 : 집합과 연산

```
>>> s1 = {1, 2, 3, 4, 5, 6}
```

```
>>> s2 = {4, 5, 6, 7, 8, 9}
```

```
>>> s1.union(s2) # 합집합을 구함
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
>>> s1.intersection(s2) # 교집합을 구함
```

```
{4, 5, 6}
```

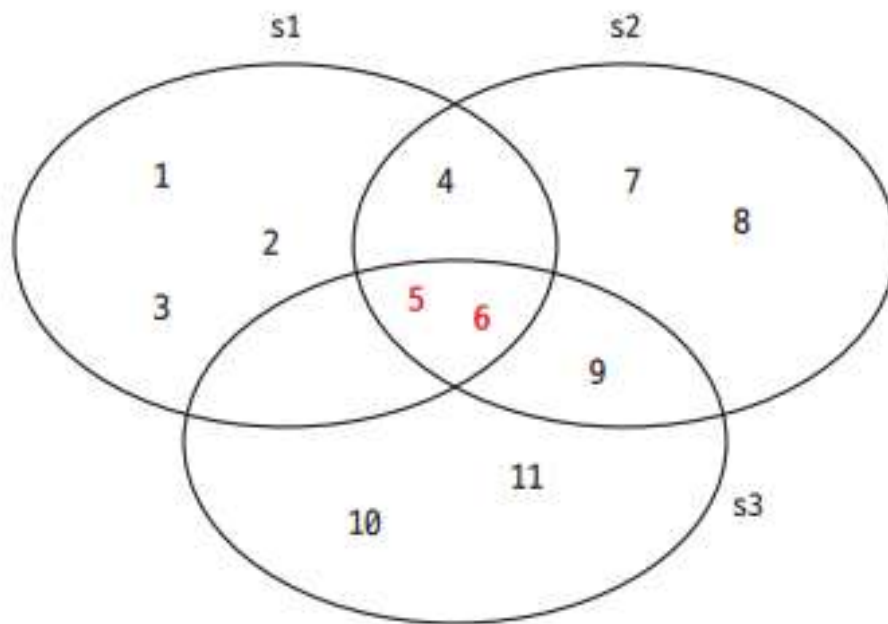
```
>>> s1.difference(s2) # 차집합을 구함
```

```
{1, 2, 3}
```

```
>>> s1.symmetric_difference(s2) # 대칭 차집합을 구함
```

```
{1, 2, 3, 7, 8, 9}
```

- 여러 개의 집합에 대한 연속적인 연산도 가능



[그림 6-9] $s1 \& s2 \& s3$ 연산의 결과

대화창 실습 : 집합과 $s1 - s2 - s3$ 연산

```
>>> s1 = {1, 2, 3, 4, 5, 6}
```

```
>>> s2 = {4, 5, 6, 7, 8, 9}
```

```
>>> s3 = {5, 6, 9, 10, 11}
```

```
>>> s1 & s2 & s3      # s1, s2, s3의 교집합을 구함
{5, 6}
```

```
>>> s1 - s2 - s3      # s1에서 s2, s3의 원소를 뺀다
{1, 2, 3}
```

대화창 실습 : 부분집합과 상위집합 연산

```
>>> s1 = {1, 2, 3, 4, 5}
>>> s2 = {1, 2, 3}      # s1의 부분집합임
>>> s3 = {1, 2, 6}      # s1의 부분집합이 아님
>>> s2.issubset(s1)     # s2가 s1의 부분집합인지를 묻는 메소드
True
>>> s3.issubset(s1)     # s3이 s1의 부분집합인지를 묻는 메소드
False
>>> s1.issuperset(s2)   # s1이 s2의 상위집합인지를 묻는 메소드
True
>>> s1.issuperset(s3)   # s1이 s3의 상위집합인지를 묻는 메소드
False
```

대화창 실습 : 두 집합의 서로소 연산

```
>>> s1 = {1, 2, 3}
>>> s2 = {10, 20, 30}
>>> s1.isdisjoint(s2)   # 집합 s1이 s2과 서로소 인가 검사
True
```

case 1. 어디에 저장할까?

문자열 1: Hello, world!
문자열 2: how are you?
공통 문자: d l
문자열 1에만 출현한 문자: d l

문장: Apple: "Papa!"
출현한 알파벳: e l a p

case 2. 어디에 저장할까?

동윤이가 새로 문을 열고 운영하는 커피 가게에는
Americano, Ice Americano, Cappuccino, Caffe Latte,
Espresso의 5가지 메뉴가 있으며, 각 메뉴의 가격은 각각
3,000원, 3,500원, 4,000원, 4,500원, 3,600원이다.

- 어디에 저장할까?

```
lst = [3000, 3500, 4000, 4500, 3600]
```

- 가격은 저장했지만, 어떤 음료의 가격인지는 알 수 없음
- 데이터의 의미 누락됨

아메리카노의
가격은??

case 3. 어디에 저장할까?

사용자가 'stop'을 입력할 때까지 반복하여 문장을 입력받아, 문장에 포함된 알파벳의 빈도를 출력하시오.

- 대소문자를 무시하고 카운트할 것.

```
문장: List  
문장: Str  
문장: stop
```

```
l: 1  
i: 1  
s: 2  
t: 2  
r: 1
```

바구니에 든 과일의 개수 세기

```
basket = ['사과', '바나나', '사과', '포도', '바나나', '사과']  
cnt = [3, 2, 1] ????
```

딕셔너리 dictionary

딕셔너리의 정의와 멤버 접근, 추가, 삭제

딕셔너리에 사용 가능한 built-in ft. 연산자(in, not in, ==, !=)

딕셔너리에서 제공하는 메소드

keys(), values(), items(), get(), pop(), popitem(), clear()

리스트와 비교

언제 사용하나?

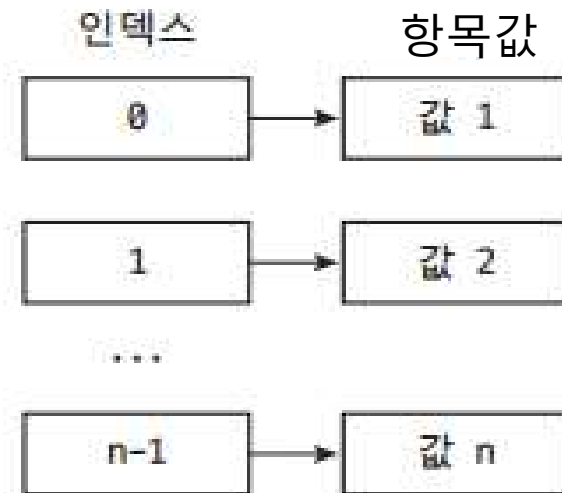
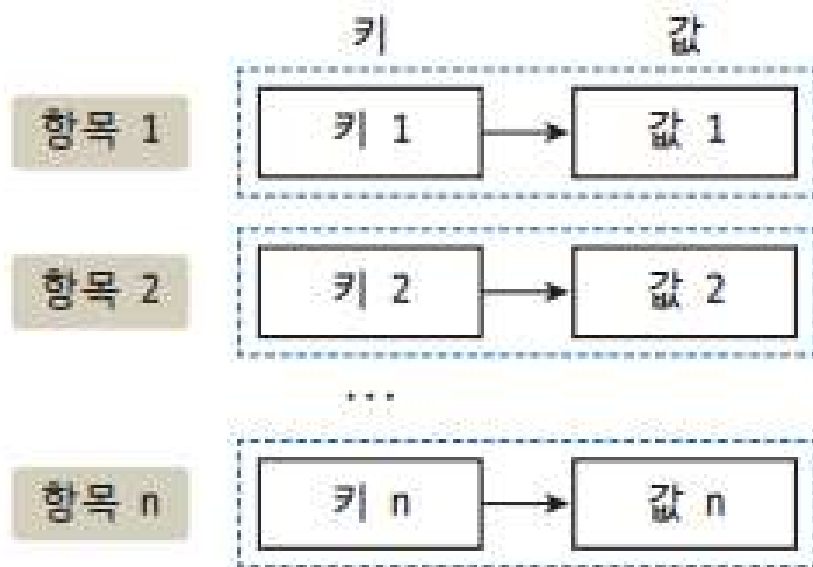
딕셔너리

- 키key와 값value을 쌍으로 갖는 자료형
- 키를 이용하여 값을 참조
- 필요성
 - 아래 그림과 같이 키:값의 쌍으로 이루어진 정보

Americano	가격 : 3,000원
Ice Americano	가격 : 3,500원
Cappuccino	가격 : 4,000원
Cafe Latte	가격 : 4,500원
Espresso	가격 : 3,600원

키	값
2019001	'이순신'
2019002	'김유신'
2019003	'강감찬'

딕셔너리 vs. 리스트



리스트의 인덱스와 항목

- 항목에 접근하는 방법
 - 딕셔너리:
 - 리스트:

[표 6-1] person 딕셔너리의 항목과 키^{key}-값^{value}

항목	키	값
항목1	'이름'	'홍길동'
항목2	'나이'	27
항목3	'몸무게'	82

대화창 실습 : hong 딕셔너리의 생성과 조회

```
>>> hong = {'이름' : '홍길동', '나이' : 26, '몸무게' : 82 }
```

```
>>> hong['이름']
```

```
'홍길동'
```

```
>>> hong['나이']
```

```
26
```

```
>>> hong['몸무게']
```

```
82
```

- 3명의 학생에 대한 학번 : 이름의 쌍을 가지는 새로운 딕셔너리 students를 생성

[표 6-2] students 딕셔너리의 항목과 키key-값value

항목	키	값
항목1	2019001	'이순신'
항목2	2019002	'김유신'
항목3	2019003	'강감찬'

대화창 실습 : students 딕셔너리의 생성과 조회

```
>>> students = { 2019001:'이순신', 2019002:'김유신', 2019003:'강감찬'}
```

```
>>> students[2019001]
```

```
'이순신'
```

```
>>> students[2019002]
```

```
'김유신'
```

```
>>> students[2019003]
```

```
'강감찬'
```

2. 다음 키-값의 항목을 요소로 가지는 fruits_dic 딕셔너리를 생성하시오. 그리고 이 딕셔너리를 이용하여 각 과일의 가격을 다음과 같이 출력하여라.

키	값
'apple'	5000
'banana'	4000
'grape'	5300
'melon'	6500

apple의 가격은 5000원입니다.

banana의 가격은 4000원입니다.

grape의 가격은 5300원입니다.

melon의 가격은 6500원입니다.

딕셔너리의 삽입과 삭제

- 딕셔너리에 새로운 항목을 삽입하려면 다음과 같은 방법 사용

```
딕셔너리이름[키] = 값
```

- 삭제 시 del 키워드 다음에 삭제할 딕셔너리 항목의 키를 입력
- 존재하지 않는 키를 이용하여 항목을 삭제하려하면 **KeyError**라는 오류메시지가 출력됨

대화창 실습 : 딕셔너리의 항목 삽입

```
>>> hong = {'이름' : '홍길동', '나이' : 26, '몸무게' : 82}
>>> hong['직업'] = '울도국의 왕'  # 새로운 키:값의 항목 삽입
>>> hong
{'이름': '홍길동', '나이': 26, '몸무게': 82, '직업': '울도국의 왕'}
```

대화창 실습 : 딕셔너리의 항목 수정

```
>>> hong = {'이름' : '홍길동', '나이' : 26, '몸무게' : 82}
>>> hong['나이'] = 27
>>> hong
{'이름': '홍길동', '나이': 27, '몸무게': 82}
```

대화창 실습 : 딕셔너리의 항목 삭제

```
>>> hong = {'이름' : '홍길동', '나이' : 26, '몸무게' : 82 }  
>>> del hong['나이']  
>>> hong  
{'이름': '홍길동', '몸무게': 82}
```

대화창 실습 : 딕셔너리의 항목 삭제 - KeyError 오류 발생

```
>>> hong = {'이름' : '홍길동', '나이' : 26, '몸무게' : 82 }  
>>> del hong['출생지']  
....  
KeyError: '출생지'
```

딕셔너리를 위한 함수와 연산

- len()
 - 딕셔너리 내의 항목의 수를 얻기 위해 사용
- sum(), max(), min()
 - 키를 대상으로 함
- in , not in
 - 키와 값이 딕셔너리에 있는지 확인
- ==, !=
 - 두 개의 딕셔너리가 동일한 항목을 가지고 있는지 확인

대화창 실습 : 딕셔너리의 항목 삽입

```
>>> hong = {'이름' : '홍길동', '나이' : 26, '몸무게' : 82}
```

```
>>> len(hong)    # 딕셔너리 항목의 개수를 반환
```

```
3
```

```
>>> '이름' in hong # '이름'은 키에 있음
```

```
True
```

```
>>> '직업' in hong # '직업'은 키에 없음
```

```
False
```

```
>>> '홍길동' in hong # '홍길동'은 값에 있으나 키에 없음(주의)
```

```
False
```

- 딕셔너리에서는 >, >=, <, <=와 같은 비교연산을 지원하지 않는다.

대화창 실습 : 딕셔너리의 비교 연산

```
>>> d1 = {'이름' : '홍길동', '나이' : 26 }
```

```
>>> d2 = {'나이' : 26, '이름' : '홍길동' }
```

```
>>> d1 == d2
```

```
True
```

```
>>> d1 > d2
```

```
....
```

```
TypeError: '>' not supported between instances of 'dict' and 'dict'
```

딕셔너리의 메소드

메소드	하는 일
keys()	딕셔너리 내의 모든 키를 반환한다.
values()	딕셔너리 내의 모든 값을 반환한다.
items()	딕셔너리 내의 모든 항목을 [키]:[값] 쌍으로 반환한다.
get(key)	키에 대한 값을 반환한다. 키가 없으면 None을 반환한다.
pop(key)	키에 대한 값을 반환하고, 그 항목을 삭제한다. 키가 없으면 KeyError 예외를 발생시킨다.
popitem()	마지막 항목을 반환하고 그 항목을 삭제한다.
clear()	딕셔너리 내의 모든 항목을 삭제한다.

대화창 실습 : 딕셔너리의 keys(), values(), items() 메소드

```
>>> person = {'이름' : '홍길동', '나이' : 26, '몸무게': 82}
```

```
>>> person.keys()
```

```
dict_keys(['이름', '나이', '몸무게'])
```

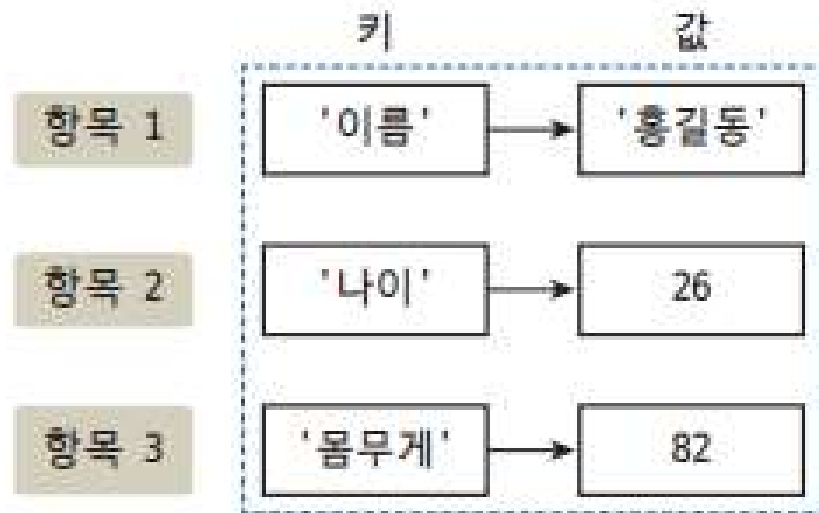
```
>>> person.values()
```

```
dict_values(['홍길동', 26, 82])
```

```
>>> person.items()
```

```
dict_items([('이름', '홍길동'), ('나이', 26), ('몸무게', 82)])
```

- person.items() 메소드를 통해 반환되는 항목들은 ('이름', '홍길동')과 같이 괄호내에서 쉼표로 구분되어 있음
- 이러한 자료형을 **튜플tuple**이라고 함



[그림 6-3] person 딕셔너리 항목 내의 키^{key}와 값^{value}

- person['취미']를 통해서 '취미'를 키로 하는 값을 가져오자.
- '취미' 키가 person의 항목에 들어 있지 않기 때문에 KeyError라는 예외를 발생시키고 프로그램은 중지됨.
- 이를 방지하기 위해서 우리는 get() 메소드를 사용
- get() 메소드는 키를 인자로 넣어서 이 키가 딕셔너리에 있을 경우 값을 반환하며, 키가 없을 경우 None이라는 값을 반환

대화창 실습 : 딕셔너리의 get() 메소드

```
>>> person = {'이름' : '홍길동', '나이' : 26, '몸무게': 82}
>>> person['취미']    # '취미' 키가 없으므로 여기서는 예외를 발생시킴
...
KeyError: '취미'
>>> person.get('취미') # '취미' 키가 없으므로 None을 반환함
>>> person.get('이름')
'홍길동'
```

대화창 실습 : 딕셔너리의 메소드 살펴보기

```
>>> person = {'이름' : '홍길동', '나이' : 26, '몸무게': 82}
>>> person.popitem()      # 마지막 항목 삭제 후 반환
('몸무게', 82)
>>> person                # 랜덤하게 선택된 '몸무게' 항목이 삭제되었으므로 2개 항목만 있음
{'이름': '홍길동', '나이': 26}
>>> person.pop('나이')    # '나이' 항목을 반환한 후 삭제함
26
>>> person                # '나이' 항목이 삭제되었으므로 1개 항목만 있음
{'이름': '홍길동'}
>>> person.clear()        # person 딕셔너리의 모든 항목을 삭제함
>>> person
{}

```

- 딕셔너리의 항목을 하나하나 열거하기 위해서 for - in문을 사용
- 리스트의 for - in 문이 하나의 항목을 반환하는 반면 딕셔너리의 for - in 문은 **항목의 키를 반환**한다는 특징이 있음

대화창 실습 : for 문을 이용한 딕셔너리의 순회

```
>>> person = {'이름' : '홍길동', '나이' : 26, '몸무게' : 82 }
```

```
>>> for key in person:
```

```
...     print('{} : {}'.format(key, person[key]))
```

```
...
```

```
이름 : 홍길동
```

```
나이 : 26
```

```
몸무게 : 82
```

리스트와 딕셔너리의 비교

```
lst = [5, 10, 15, 20, 25]
print(lst.pop(0))
print(lst)
print(lst[1])
```

```
dct = {0:5, 1:10, 2:15, 3:20, 4:25}
print(dct.pop(0))
print(dct)
print(dct[1])
```

```
colors = ('red', 'green', 'blue')
values = ('#FF0000', '#008000', '#0000FF')
```

```
{('red', '#FF0000'), ('green', '#008000'), ('blue', '#0000FF')}
```

```
items = ['옷', '컴퓨터', '모니터']
prices = [100000, 2000000, 320000]
```

```
store = (('옷', 100000), ('컴퓨터', 2000000), ('모니터', 320000))
```

```
colors = ['red', 'green', 'blue']
values = ['#FF0000', '#008000', '#0000FF']
```

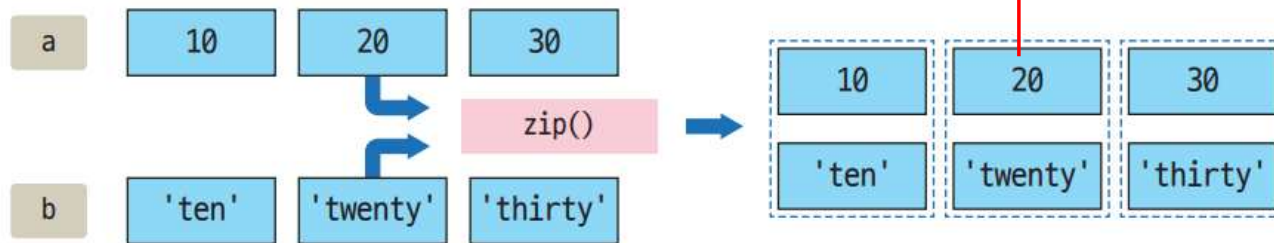
```
c_dct = {'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

zip() 함수를 이용한 집적화

대화창 실습 : zip() 함수를 이용한 집적화

```
>>> a = [10, 20, 30]           # 리스트 a
>>> b = ('ten', 'twenty', 'thirty') # 튜플 b
>>> for val in zip(a, b): # 리스트 a와 튜플 b를 집적화하여 생성된 튜플을 출력
    print(val)
```

```
(10, 'ten')
(20, 'twenty')
(30, 'thirty')
```



[그림 6-11] 리스트 a와 튜플 b의 zip() 함수를 이용한 집적화와 그 결과 튜플

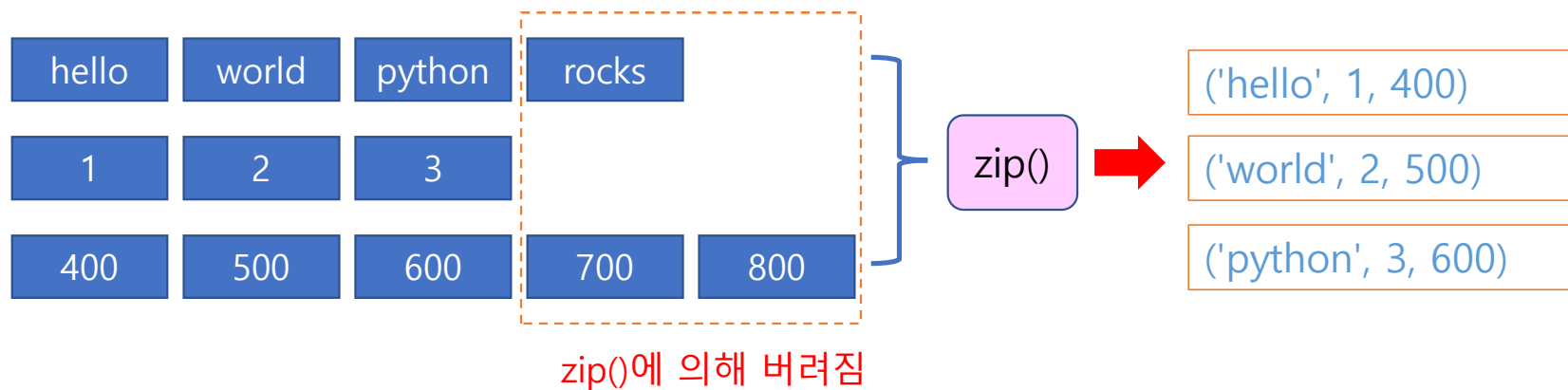
zip() 함수를 이용한 집적화(cont.)

- 리스트, 딕셔너리, 집합, 튜플과 같은 자료형을 **반복가능** **iterable** 자료형이라고 한다.
- 반복가능 자료형 여러 개를 넘겨주면, 이들을 **합쳐서** 튜플 **반복자** **iterator**를 반환하는 함수가 zip() 함수이다.
- zip() 함수는 여러 개의 반복가능 자료형을 받을 수 있다. 이를 **집적화** **aggregation**라고 한다.
- 다수의 반복가능 자료형 인자 n 개가 들어오면 튜플 반복자는 인자의 수인 n 개 만큼의 튜플을 원소로 가진 튜플 반복자가 됨.
- 이때 i번째 원소인 튜플은 인자로 넘어온 각 반복가능 자료형 인자의 i번째 원소들로 구성됨.

- 여러 개의 반복가능 자료형 인자를 전달한 경우

대화창 실습 : zip() 함수에 세개의 반복가능 자료형 인자를 전달

```
>>> str_list = ['hello', 'world', 'python', 'rocks']  
>>> int_tuple = (1, 2, 3)  
>>> int_list = [400, 500, 600, 700, 800]  
>>> my_iterator = zip(str_list, int_tuple, int_list)  
>>> list(my_iterator)  
[('hello', 1, 400), ('world', 2, 500), ('python', 3, 600)]
```



zip() 되돌리기

- zip() 함수를 통해 묶여진 튜플들을 원래의 반복가능 자료들로 되돌리는 작업
- 다른 함수를 쓰는 것이 아니라 zip() 함수에 묶인 자료를 인자로 입력하고 * 연산자를 인자 앞에 붙이기

대화창 실습 : zip()으로 묶여진 튜플을 다시 풀어내는 방법(언패킹)

```
>>> zip_lst = list(zip(str_list, int_tuple, int_list))
>>> a, b, c = zip(*zip_lst)    # 튜플을 언패킹 함
>>> print(a, b, c)
('hello', 'world', 'python') (1, 2, 3) (400, 500, 600)
```

Remark

- 하나의 반복가능 자료형 인자가 들어올 경우 zip() 함수는 하나의 튜플에 대한 반복자를 반환

대화창 실습 : zip() 함수에 하나의 반복가능 자료형 인자를 전달한 경우

```
>>> lst = ['a', 'b', 'hello', 'this']  
>>> my_iterator = zip(lst)  
>>> result = set(my_iterator)  
>>> result  
{('a',), ('b',), ('hello',), ('this',)}
```